OS Project - Team K

2026/01/22
Ahmad Shami
Tatiana Khoodaruth
Viktoriia Shtanko
Andrej Zlatevski

Table of contents

Table of contents……1
Implemented functionalities……1
User specifications……1
User Tests……...5
Implementation……6
Team organization……...10

Implemented functionalities

Our NachOS allows the user to read and write characters or strings from the terminal through a synchronized console. The user is also able to create and join user threads in a given process. There's a limited number of threads per process that can be handled which is relative to the UserStackSize and UserStackSegment. User threads automatically terminate in the current context by calling UserThreadExit. Semaphore, condition variables and mutex locks are available in both the user space and the kernel space to allow synchronization. The user is able to launch new processes on the system within those newly created processes, multiple threads can be managed or launch child processes. There are two versions of the memory allocator that are available, one based on the memory allocator done in the lab and the second one based on a memory allocator found online. The user is also able to receive or send messages of fixed or variable sizes using a reliable networking layer and ***experimentally*** can send files over the network and even offload processes to other machines. Concerning the filesystem, the user can manipulate files with a variable size, read and write data from them and also make use of directories in the standard UNIX way using absolute and relative paths.

User Specifications

void Halt(): A function to stop NachOS and print out performance stats. This function should be called at the end of a user program's `main` function.
void PutChar(char ch): Displays character ch on the console.
int GetChar(): saves a character from the console and returns it .
void PutString(char *s): outputs a string located in user memory to the console. At most MAX_STRING_SIZE characters are read. The operation is thread-safe and terminates on the first '\0' encountered.

char * GetString(char *s, int n): Reads characters from the console into a buffer located in user memory, stopping on newline, end-of-file, or after n−1 characters. The resulting string is always null-terminated. The operation is thread-safe.
void PutInt(int n): Outputs the decimal representation of the integer n to the console. The integer is converted in kernel space and printed character by character. The operation is thread-safe.
int GetInt(int *n): Reads a sequence of numeric characters from the console and converts it to an integer. Reading stops at the first non-numeric character. If no valid number is read, *n is set to 0. The operation is thread-safe.
int feof(): A function call that returns whether an `EOF` has been detected on the console or not. useful for handling automated inputs or file streams.
int UserThreadCreate(void (*f)(void *), void *arg): Creates a new user thread in the current process that starts execution at address entry with a unique ID during live execution of the current process. The function pointer f and argument arg are passed to the new thread. The new thread shares the address space of the calling process. Returns a positive thread identifier on success, or -1 if thread creation fails. This call automatically adds a termination of the thread by calling UserThreadExit at the end of the function call.
void UserThreadJoin(int id): Halts the calling function until either thread with thread ID specified finishes execution, or waits for any threads to exit if the parameter ID is equal to USERTHREAD_JOIN_ALL(-1).
void UserThreadExit(): Terminates the calling user thread. Any thread waiting in UserThreadJoin on this thread is released. When the last user thread exits, a pending UserThreadJoin(-1) is unblocked. This call does not terminate the entire process unless it is the last thread (main thread of the process).
Note: Must include "libthread.h" library
void UserThreadRoot(void (*f)(void *), void *arg): A wrapper to execute a function followed directly by **UserThreadExit** call. This is used to implement automatic thread termination invoked by the **UserThreadCreate** syscall.
Note: Must include "libthread.h" library
int SemInit(int semVal): Creates a new user-level semaphore (mapped to a kernel semaphore) with initial value semVal. Returns a non-negative semaphore identifier on success. The semaphore is visible to all threads in the process.
int SemWait(int sem_id): Performs a wait (P) operation on the semaphore identified by sem_id. Blocks the calling thread if the semaphore value is zero. Returns 0 on success, or -1 if the semaphore identifier is invalid.
int SemPost(int sem_id): Performs a signal (V) operation on the semaphore identified by sem_id, potentially waking a waiting thread. Returns 0 on success, or -1 if the semaphore identifier is invalid.
int SemDestroy(int sem_id): Destroys the semaphore identified by sem_id and releases associated kernel resources. Returns 0 on success, or -1 if the semaphore identifier is invalid.
int ForkExec(char *executable): Creates a new process, which will run the program specified by the executable name. In case of insufficient physical memory, it returns -1. Otherwise, it creates a new address space, it assigns it to the new kernel thread created who will be seen by the scheduler as a separate executable context. It then returns the PID of the newly created process. The process creation is done atomically, with interrupts off and it is thread safe.

void ProcessJoin(int pid): The calling process is suspended until the process with the specified PID finishes execution. If PID = PROCESS_JOIN_ANY, the process is suspended until any process finishes execution, regardless of the PID.
Exit(int status): The calling process terminates, freeing physical frames in the main memory as intended and notifies the processes who wait on the termination of the process.
void KDEBUG(char flag, char * msg): An interface to kernel **DEBUG** function (similar to demsg on unix)
void* Sbrk(): Returns the brk pointer which points to the beginning of the heap section.
unsigned int GetProcessHeapSize(): returns the heap size of the heap section.
int Create(char* path, int size): Creates a file of specific size and names it based on the "path" argument. The argument can be just a file name, in which case the file will be created in the current directory, or it can be a relative/absolute path, indicating in which directory the file should be created. In the latter part the file name is extracted by the last chunk of string, delimited by "/". Returns 1 if the operation is successful, 0 otherwise.
int Open(char* path): Opens a file specified by "path" (if it exists) and returns its corresponding file descriptor.
int Write(char* buffer, int size, int fd): Writes "size" bytes from the string buffer to the file specified by the file descriptor. Returns the number of bytes actually written.
int Read(char* buffer, int size, int fd): Read "size" bytes from the file specified by the file descriptor and write the data in the buffer. Returns the number of bytes actually read.
void Close(const char* path): Closes the file specified by the argument "path". It is not a UNIX complaint as normally the file descriptor is used. The file will be closed from reading and writing operations.
int Remove(char* path): Deletes the file from the filesystem specified by the "path" argument. Returns 1 if the operation is successful, 0 otherwise.
void List(): Lists the contents of the current directory.
int CreateDirectory(char* path): Creates a directory either by using absolute/relative paths or just by a directory name. Returns 1 if successful, 0 otherwise.
int ChangeDirectory(char* path): Changes the current directory to the one specified by "path". Returns 1 if successful, 0 otherwise.
int DeleteDirectory(char* path): Deletes the directory specified by "path". Returns 1 if successful, 0 otherwise.
#A more friendly user interface of rnet is provided via rnet userspace library (rnet.h)
struct RNetArgs: structure that holds up to 6 register values to overcome MIPS limitations (pass arguments in memory rather than registers)
int RNetSocket(RNetArgs* a): Returns a socket descriptor to userspace.
int RNetClose(RNetArgs* a): closes a socket via its descriptor.
int RNetSendFixed(RNetArgs* a): sends a fixed sized packet via a socket.
int RNetRecvFixed(RNetArgs* a): receives a fixed sized packet via the socket.
int RNetSendVar(RNetArgs* a): sends a variable sized packet via a socket.
int RNetRecvVarBegin(RNetArgs* a): starts the variable packet received by returning metadata (length, src machine, src box).
int RNetRecvVarRead(RNetArgs* a): returns the actual packets of data to the caller.

int RNetRecvVarEnd(RNetArgs* a): clears the kernel buffer used to receive variable data to allow other transmissions over the same socket.
int Migrate(int dstMachine): migrates the process to the destination machine.

libthread userspace library:

struct sem_t: a structure containing the identifier of the kernel managed semaphore.
int UserThreadCreate(void (*f)(void *), void *arg): As described in the previous part. The implementation is bound to this library.
int sem_init(sem_t * sem, int val): Initializes the semaphore structure with the desired initial value "**val**"
int sem_wait(sem_t * sem): Waits on the semaphore. A wrapper around **SemWait** syscall.
int sem_post(sem_t * sem): Posts the semaphore. A wrapper around **SemPost** syscall.
int sem_destroy(sem_t * sem): Destroys the kernel semaphore. A wrapper around **SemDestroy** syscall.
Other implemented features like user space semaphores and locks are mentioned in the appendix

mem-alloc userspace library:

void *memory_alloc(SizeT size): Given a size, allocates a chunk of requested size from the heap(based on first fit policy). Returns a pointer to the allocated block on success else returns NULL.
void memory_free(void *p): Given a pointer, frees the memory at that space. Upon double freeing, it will lead to memory leaks.
void *memory_init(void): initializes the pointer to beginning of the heap by invoking **Sbrk()** systemcall.

liballoc userpace library: an alternative more tested heap allocator (POSIX compliant)
More details about this library in the appendix

rnet userspace library (rnet.h):
int rnet_socket(int localMailbox): Opens a socket (handle in the kernel) and returns the descriptor of that socket.
int rnet_close(int h): Closes the socket (cleans the handle in the kernel) and returns status (success: 0, fail: -1).
int rnet_send_fixed(int h, int dstM, int dstB, const void* buf, int len): Sends fixed data packet and returns the status.
int rnet_recv_fixed(int h, void* buf, int cap, int* srcM, int* srcB): Received fixed sized data and returns the number of bytes received (or -1 on fail).
int rnet_send_var(int h, int dstM, int dstB, const void* buf, int len): send variable sized data and return the status of the send.
int rnet_recv_var_begin(int h, int* outLen, int* srcM, int* srcB): starts the receiving process of variable sized data and returns the status.

int rnet_recv_var_read(int h, void* buf, int cap): copy the received data from the kernel and return the number of bytes copied/received returns -1 on failure.
int rnet_recv_var_end(int h): cleans the handle data field to allow other data received.
int rnet_recv_var(int h, void* buf, int cap, int* outLen, int* srcM, int* srcB): a wrapper around the 2-phase recvvar to make it more convenient for the user. In case the peers can negotiate how large the user buffer should be (for example for file transfer, report how large the file is beforehand). Returns -1 on failure or number of bytes received on success.

User Tests (code/test folder)

Brief description of our tests:
getchar.c: Reads one character given as input from keyboard and prints it on the console.
getstring.c: Reads a string given as input from the keyboard and prints it on the console. String size limited to 20.
getint.c: Reads an integer as input and displays that integer incremented by 5.
putchar.c: Prints 4 characters on the console corresponding to 'abcd'
putint.c: Prints an integer to the console.
putstring.c: Prints 2 strings to the console.
prod_cons_test.c: Using a buffer of size 5, producer function will produce a total of 10 elements and consumer function will consume a total number of 10 elements from the buffer. That critical section (buffer) can only be accessed by one thread at a time using user semaphores.
single_join.c: creates one user thread, waits for it using UserThreadJoin, and terminates only after the worker thread exits
many_join.c: Creates multiple user threads, each printing its identifier and exiting.The main thread waits for all worker threads using UserThreadJoin.
all _join.c: A main thread waits for all child threads to finish before continuing.
no _join.c: The opposite, here the main thread does not wait, therefore leading to erroneous code (by the programmer using the library).
createmanythreads .c: Creates 100 threads, testing the limits of NachoOS (depending on the UserStackSize value and also the total physical memory).
threads_create_threads.c: Because of memory limitations, each thread creates a thread. The goal of this test is to see if any issues would arise if a programmer tries to code this specific hierarchical scenario.
launch _userpages.c: This test creates 2 processes, where each process creates a thread and prints a string to the console. This is the initial test based on the assignment sheet.
launch_many _processes.c: Test similar as the previous one, only this time we predefined the number of processes NachOS launches. The processes also launch a bigger number of threads, each of them doing the same, i.e printing letters to the console. We even tested with 100 processes, with the initial size of the main memory, and NachOS runs perfectly fine.
minishell.c: This test is inspired by the test/shell.c file. It is essentially a shell where users can type which test program to launch, either in foreground or in background. Writing "exit" halts NachOS. The user also can manipulate the filesystem using similar UNIX syntax. For more information, the user can receive the command list while writing "help". An unfortunate bug is found late where it is not possible to run

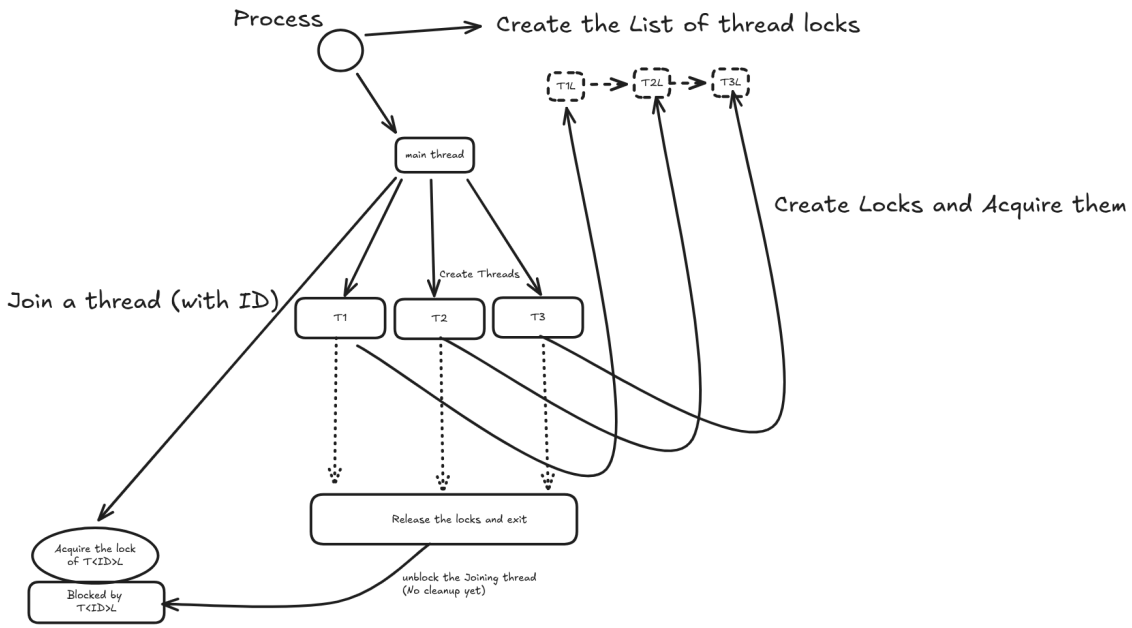
background processes using the FILESYS mode. However, that is not true if the minishell is run in FILESYS-STUB mode.
mem_alloc_stress_test.c: This test allocates two int pointers and one char pointer, initializes elements in both int pointers and frees them. If we uncomment where we try to allocate an int pointer p4 larger than the process address space, the process will crash (this bug is not handled).

Folders userprog and filesys
userprog/dirtest.cc: The specified test in kernel space is similar to the fstest.cc. The goal of this test is to dynamically test our filesystem i.e creating multiple files, creating subdirectories, writing data to files, displaying and deleting them. Every step is documented using the respecting DEBUG flags.
filesys/fstest.cc: This test was given by the initial NachOS developers and it is quite a useful one. It essentially stresses the system while writing big chunks of data to a file and displays statistics.

Beta Testing branch (part6):
rnet_pingpong.c: This test checks the basic functionalities of the network by sending a message and receiving a corresponding message for N iterations for fixed sized messages.
rnet_file_server.c: This test will connect to a file client, wait for the client to request its file and send the file content in the specified path by the client
rnet_file_client.c: This test will connect to a file server, request a file, give the path where the client wants to save it.
migration_demo.c: A program that tests the migration of a process from one machine to another.

Implementation

Thread Implementation:



#note: Did not implement UserThreadJoin based on the POSIX standard for simpler implementation.

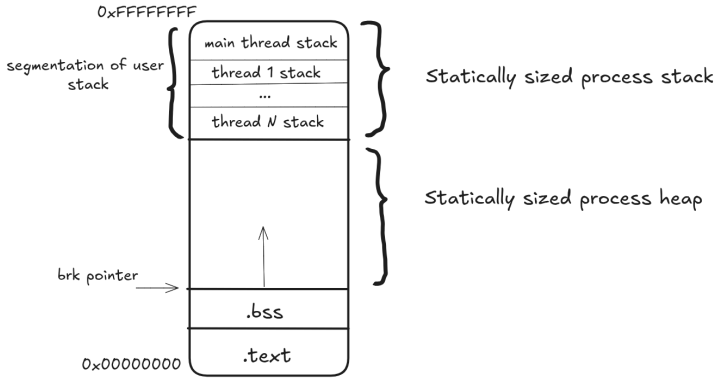
In order to manage threads of a process, we have added a Bitmap attribute to Addrspace.cc called *userStackSegmentMap* allowing us to create at most N threads including the main thread (N = UserStackSize / UserStackSegment). This bitmap allows us to know how many threads have been created for one process and the placement in this bitmap defines the thread number with 0 being reserved to the main thread. The bitmap is also used relative to a pointer variable to point to a new stack segment when a new thread is created. Additionally, we added a list called *ListThreadJoinLock* in userthreads.cc in order to track joinable user threads and allow UserThreadJoin(tid) to block until a specific user thread exits. If tid = -1, then the main thread waits for any threads to exit before it continues.

Process Implementation:

The implementation of this part lies around how ReadAtVirtual is written, how the address spaces interact and also how the frame provider works. Essentially, the constructor of AddrSpace is modified to take care of reading into virtual addresses instead of directly into the main memory. The frame provider implemented is a live global variable that lives throughout the lifecycle of NachOS visible to all processes. The class Bitmap is used again in order to track which memory segments are occupied in the main memory. The creation of a process is a similar procedure as creating a thread, only this time each process has a separate address space. This encourages atomicity when creating a new address space as the main memory is shared between processes. When there are no processes to be run in the system, NachOS halts and terminates. There exists a possibility to wait for process termination as well by calling **ProcessJoin**. We use a global process table, where each entry corresponds to a process (with fields as the PID, exit status and flags). The process creation with **ForkExec** also takes care of automatic allocator initialization (*More details about this process in the appendix*).

Memory Allocation Implementation:

We managed to implement a heap segment inside each process on its creation (details about that in the appendix). To assist the user with heap memory management, we included two heap allocators. The heap segment is of static size similar to the stack of each process. The layout of the address space is as follows:

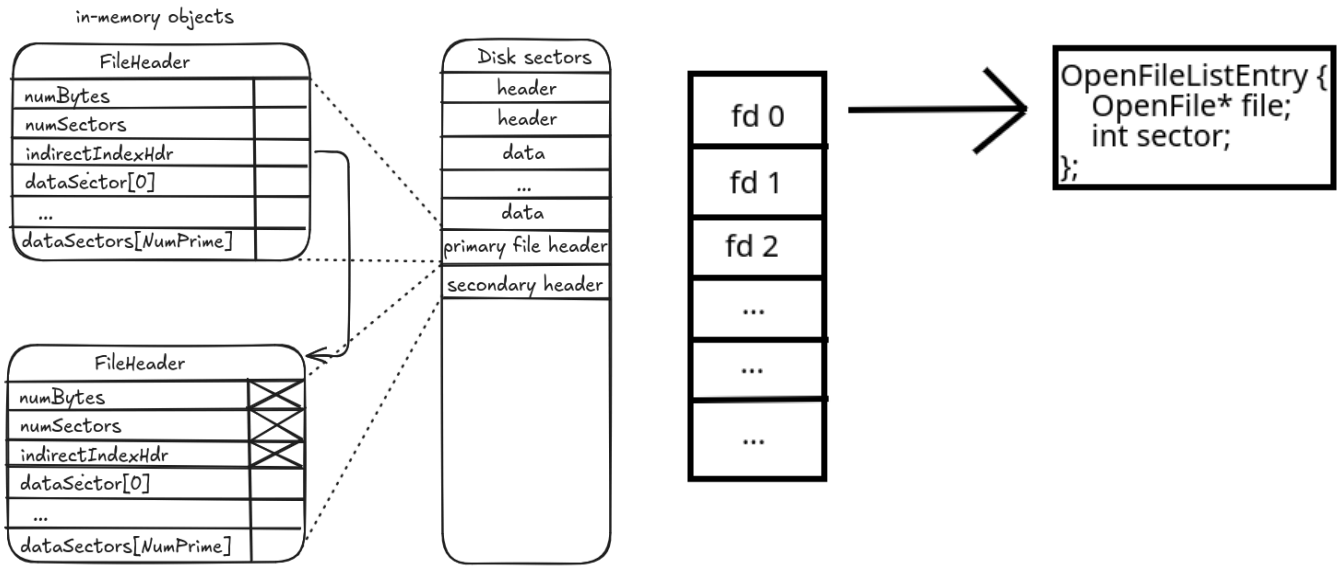


Filesystem Implementation:

Implementing the filesystem required some thinking about how the system tracks open files, what data structures to use, and how to maximize simplicity. What needs to be acknowledged here are 3 separate instances: A current directory instance, which is a pointer to a Directory object and is initialized at boot

time. The initial directory is always the root directory, in which its file header is stored in sector 1. Then we have an instance of OpenFile (named directoryFile) which points to the current file instance of the current directory, required to write back data to the disk whenever a change is made. The pointers of this instance change whenever we change the directory. And finally, a list of open files, following a similar philosophy of the UNIX specifications. The indexes represent the file descriptors and in each list entry there are 2 fields - A pointer to its corresponding OpenFile instance and a sector field, in order to track where the file header of that entry is stored in disk. A separate convention for parsing of path files used as mentioned below.

In order to increase the maximum file size, we have also implemented a two-level index based on the FileHeader class with an additional variable indirectIndexHdr pointing to a secondary file header storing an array of additional (secondary) sector indices of the actual data sectors that belong to the file. When fetching the primary file header of a file from the disk, a check of its indirectIndexHdr is performed - if it is null, no secondary array is used for the file; otherwise we're fetching the existing secondary file header from the dataSectors[NumPrime – 1] disk sector.
NumPrime (NumDirect - 1) is the number of data sectors used for storing sector indices of the actual file data sectors. The last element in dataSectors stores the sector index of the secondary file header.



Implementing dynamic sector allocation - allowing files to grow physically on the disk when processing write requests - required some modifications to the WriteAt method of the FileHeader class - whenever the resulting file size exceeds the number of sectors already allocated, the ExtendFile method of the fileSystem is called. This new method calls Allocate (also modified, with respect to the new two-level index for file data sectors) and - if there was enough free space - writes the modified state of freeMap to disk.

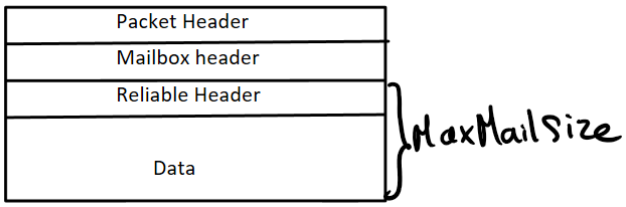
Parsing paths - which was implemented in parallel with working on the directory hierarchy in part 1 - required adding an auxiliary string parsing method (returning a list of substrings separated by the specified path delimiter in the input string). When resolving a path (finding the parent directory of the file

denoted by its relative or absolute path), we use this auxiliary method to define if the path tokens correspond to the existing subdirectories of their direct parent. The resolvePath method returns a Directory object in case there was no parsing error, and is called from all FileSystem public methods that take an argument 'path' ('name' in the initial version) to manipulate corresponding files/directories.

Network Implementation:

In order to build a reliable protocol to send data between machines we have decided to build something close to TCP but simplified based on the **send-wait protocol** where each packet needs to be acknowledged independently, it is built on top of the post class. To build ReliablePost class, it relies on a structure called PacketHeader which contains information on the type of packet that is being handled. And in order to know whether or not a connection has been established, the reliablepost structure contains a list of peerkeys to know to which machine the current machine is connected to. Each packet going through the network encapsulates a header containing the data on the receiver and the type of data in the packet.

Example of one Packet:

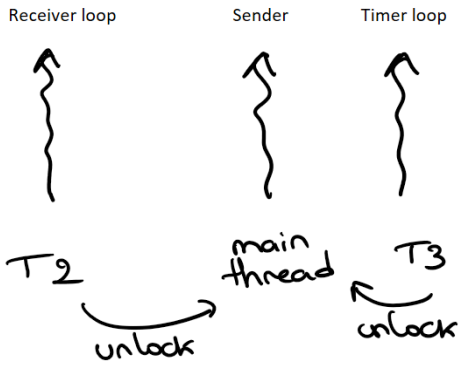


Here each packet being of a fixed size of 64, we can only fit data of size (64 - sizeof(3 headers)). The Reliable header contains a type variable to know if the packet contains data or is an ACK packet, the length of the data (0 if it's an ACK else size of data), a flag variable to know if we're dealing with fixed sized messages or messages with different sizes, a seq variable allows to track if a packet was loss or if a duplicate packet has been sent (alternating bit between 0 and 1).

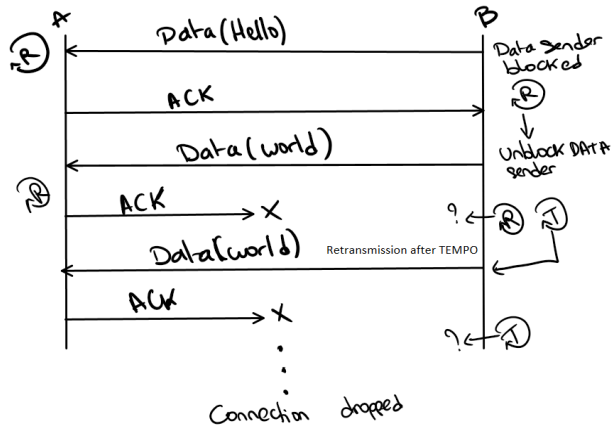
Network protocol:

In this protocol we directly start sending packets without having a prior handshake to establish the connection because we establish the connection while the first packet is being sent because this protocol is stateless. At most one data packet is being handled on the network where the packet is either acknowledged by the receiver or retransmitted after a time out. If we have machine A and machine B, when machine A sends its first packet to machine B it will establish a connection between the two machines and for each packet being sent we always check whether or not a connection exists before sending. When a packet is sent we create a pendingSend structure stored in reliablepost which will allow us to track the number ticks(TEMPO), the number of retransmissions and other useful information in case we need to retransmit that specific packet. Once the packet is sent if it's not an ACK packet, the data is stored in a list in reliablepost to be used by the receiver.

We have 3 threads in total that handle the network. We have the main thread responsible for sending the packets, the receiver worker thread responsible for retrieving and filtering the packet and the timer worker thread responsible to track the TEMPO if an ack packet hasn't been received in time or if we have reached the max number of attempts to send the packet again. Once a



packet is sent by the sender it is blocked while the receiver and timer thread take over to ensure the reception of the packet.



In this example on each data packet sent, we expect an ACK message for that packet from the receiving worker thread. Once the ACK messages have been transmitted then the receiver thread unblocks the sender thread to allow the next packet to be sent. After the sender thread sends the next packet, it will block while the receiver awaits the next ACK. If that ACK message is never delivered to the sender, the timer worker thread will take over and check if we have reached a time out and that the retransmission attempt is less than the maximum allowed, it will try to retransmit the packet until the receiver gets an ACK message. If the retransmission keeps failing, at some point the packet will be dropped and we will move on to send the next packet.

The protocol is able to handle both fixed sized and variable sized messages. Thus why in reliablepost we have two lists called fixedQ and varQ where whenever a packet of data is received, the data content will be added to the respective list based on the type of the packet (variable size messages or fixed). For variable size messages, we reconstruct the full message before adding it to varQ.

Beta Testing branch (part6):

File transfer: As a demonstration, we implemented a simple application layer protocol that allows a client to request a file from a server and write it. A detailed diagram and explanation of that is in the *Appendix below*.

Process Migration: This mechanism involves running a receiver loop (a service) always listening to mail box 1. The migrating machine will serialize the process metadata and actual pages of the process and send them, the client then de-serializes the content and populates a new address space by allocating new physical pages on the target machine. Then the registers of the process are restored from the blob, and the PC is advanced so we skip the syscall (to avoid recursive calls). Then the process on the migrating machine terminates. This migration process is demonstrated by a diagram *in the appendix*.

Team Organization

We were meeting everyday during weekdays at IM2AG between 9 am and 5:30 pm to work on the project where one of us would project their screen. During week 1 it was complicated to do parallel work thus everyone sort of worked on the same things to think collectively on the implementations. We were slightly late on week 2 on step 4, thus we planned to meet on the weekend to catch up because 2 of us got very sick during the 2nd week. The 3rd week we managed to tackle step 4 and separated in 2 teams so we

could work in parallel on the network and filesystem parts. The 4th and final week we were still working on the network and filesystem implementations, while some of us focused more on finishing the report as well as starting to structure the final presentation.

Appendix

libstr userspace library:

char* parse_ str_cmd(int index, char* in_buff, char* out_buff, int MAX_LEN): Copies the content of in_buff to out_buff, while also trimming it.
int str_cmp(const char* a, const char *b): Compares 2 strings, returns 0 they are equal.
int atoi(const char *s): Converts a string to an integer.
void strcpy(char *dst, const char *src, int cap):
int strlen(const char *s): Returns length of a string.
void rstrip_ws(char *s):
void rstrip_ws(char * s): A function that strips control characters from a string. Useful in parsing command line input for example.

libthread userspace library:

struct sem_t: a structure containing the identifier of the kernel managed semaphore.
struct mutex_lock_l: a structure containing a semaphore (**sem_t**) that is used as a mutex lock.
struct cond_t: a structure containing:

- A mutex lock **x**
- Two semaphores **s** and **h**
- A waiters counter variable

This implementation of condition variables is inspired from this paper (https://birrell.org/andrew/papers/ImplementingCVs.pdf)
int UserThreadCreate(void (*f)(void *), void *arg): As described in the previous part. The implementation is bound to this library.
int sem_init(sem_t * sem, int val): Initializes the semaphore structure with the desired initial value "**val**"
int sem_wait(sem_t * sem): Waits on the semaphore. A wrapper around **SemWait** syscall.
int sem_post(sem_t * sem): Posts the semaphore. A wrapper around **SemPost** syscall.
int sem_destroy(sem_t * sem): Destroys the kernel semaphore. A wrapper around **SemDestroy** syscall.

int UserThreadCreate(void (*f)(void *), void *arg): As described in the previous part. The implementation is bound to this library.

UserThreadCreate is implemented here to allow automatic thread termination

This is implemented by passing a routine (called **UserThreadRoot**) that will execute the function then immediately execute **UserThreadExit** afterwards. Since we need to pass a reference to that function to the syscall, we need to use inline assembly so the compiler can see that function (the assembler of **start.S** cannot see this scope. A snippet of the library code is provided below:

```
void UserThreadRoot(void (*f)(void *), void *arg) {
   f(arg);              // run user function
   UserThreadExit();    // syscall -> automatic cleanup
}

int UserThreadCreate(void (*f)(void *), void *arg) {
   int ret;
   asm volatile(
       "move $4, %1\n"   // entry = UserThreadRoot
       "move $5, %2\n"   // f
       "move $6, %3\n"   // arg
       "li   $2, %4\n"
       "syscall\n"
       "move %0, $2\n"
       : "=r"(ret)
       : "r"(UserThreadRoot),
         "r"(f),
         "r"(arg),
         "i"(SC_UserThreadCreate)
       : "$2", "$4", "$5", "$6"
   );
   return ret;
}
```

Locking mechanisms

int sem_init(sem_t * sem, int val): Initializes the semaphore structure with the desired initial value "**val**"

int sem_wait(sem_t * sem): Waits on the semaphore. A wrapper around **SemWait** syscall.

int sem_post(sem_t * sem): Posts the semaphore. A wrapper around **SemPost** syscall.

int sem_destroy(sem_t * sem): Destroys the kernel semaphore. A wrapper around **SemDestroy** syscall.

int mutex_init(mutex_lock_t * m): Initializes a user-level mutex lock using kernel managed semaphores.

int mutex_lock(mutex_lock_t * m): Acquires the mutex lock specified.

int mutex_unlock(mutex_lock_t * m): Releases the mutex lock specified.

int mutex_destroy(mutex_lock_t * m): A wrapper around **sem_destroy** to destroy the kernel managed semaphore.

int cond_init(cond_t * c, mutex_lock_t * condition_lock): Initializes a user-level condition variable using kernel managed semaphores.

int cond_wait(cond_t * c, mutex_lock_t * condition_lock): Waits on the conditional variable used utilizing a mutex lock.

int cond_signal(cond_t * c): Signals the threads held by the conditional variable specified.
int cond_broadcast(cond_t * c): Broadcasts to all threads held by the conditional variable specified.

liballoc userpace library: an alternative more tested heap allocator

void *malloc(size_t): similar to **memory_alloc** (POSIX compliant)
void *realloc(void *, size_t): POSIX compliant realloc function that resizes an allocated heap buffer pointed by the first parameter. The second parameter is the new size of the buffer.
void *calloc(size_t, size_t): POSIX compliant calloc function that allocates a buffer of the number of items equivalent to what is specified in parameter 1 and each of the size specified in parameter 2. It is initialized by zeros.
void free(void *): POSIX compliant free function that liberates a heap buffer given a pointer to that buffer.

rnet userspace library (rnet.h):
int rnet_socket(int localMailbox): Opens a socket (handle in the kernel) and returns the descriptor of that socket.
int rnet_close(int h): Closes the socket (cleans the handle in the kernel) and returns status (success: 0, fail: -1).
int rnet_send_fixed(int h, int dstM, int dstB, const void* buf, int len): Sends fixed data packet and returns the status.
int rnet_recv_fixed(int h, void* buf, int cap, int* srcM, int* srcB): Received fixed sized data and returns the number of bytes received (or -1 on fail).
int rnet_send_var(int h, int dstM, int dstB, const void* buf, int len): send variable sized data and return the status of the send.
int rnet_recv_var_begin(int h, int* outLen, int* srcM, int* srcB): starts the receiving process of variable sized data and returns the status.
int rnet_recv_var_read(int h, void* buf, int cap): copy the received data from the kernel and return the number of bytes copied/received returns -1 on failure.
int rnet_recv_var_end(int h): cleans the handle data field to allow other data received.
int rnet_recv_var(int h, void* buf, int cap, int* outLen, int* srcM, int* srcB): a wrapper around the 2-phase recvvar to make it more convenient for the user. In case the peers can negotiate how large the user buffer should be (for example for file transfer, report how large the file is beforehand). Returns -1 on failure or number of bytes received on success.

Kernel Specifications

userthread:

threadLockBundle_t *FindJoinTCB(int id): Helper function that returns a bundle based on the ID
void StartUserThread(int f): Starts a user thread by passing a bundle of the entry function to it.
int do_UserThreadCreate(int entry,int f, int arg): Syscall to create a user thread job, and the entry is a stub to allow automatic termination (facilitated by **UserThreadRoot** in **libthread.h**)
void do_UserThreadJoin(int id): UserThreadJoin: Hold the calling function until the thread with the specified ID returns or waits for any thread to finish if the id is set to USERTHREAD_JOIN_ALL (or -1)
void do_UserThreadExit(): An exit call for a single thread that also tried to release the main thread via a semaphore

frameprovider:
int FrameProvider::GetEmptyFrame(bool randomAllocation): Returns which empty frame is usable.
void FrameProvider::ReleaseFrame(int physFrame): Frees a frame to be reused.
int FrameProvider::NumAvailFrame(): Returns the number of available frames.

userprocess:
void StartUserProcess(int arg): Starts a user process given an address space.
int do_ForkExec(char *executable): Syscall handler to create a new process running on a new thread.
int FindAnyJoinableProcess(): Helper function that returns which PID is joinable.
void do_ProcessJoin(int pid): Waits for a process to terminate given its PID, or waits for any process to exit given the macro PROCESS_JOIN_ANY.
void do_Exit(int status): Cleanly exits a process and cleans up its thread.

Reliable:
void SenRaw(int dstMachine, int dstMailbox, const char *buf, int nBytes): Sends a raw packet using the post office directly
SendAck(int dstMachine, int dstMailbox, unsigned short seq): Sends an ACK with a sequence identifier.
GetOrCreatePeer(int srcMachine, int srcMailbox): Gets a peer state based on the machine/box identifier key or creates a state if new.
EnqueueFixed(int srcMachine, int srcMailbox, const char *data, int len): Add message to queue. It takes a validated, duplicate-free payload and turns it into a message that the application can safely receive later.
HandleVarFragment(PeerState *peer, int srcMachine, int srcMailbox, const char *frag, int fragLen, unsigned char flags): Helper that handles the assembly and enqueueing of variable sized messages.
HandleIncoming(int srcMachine, int srcMailbox, const char *buf, int nBytes): Parser for packets. Decides the workflow of sender based on the contents of packet parsed.
ReceiverLoop(): Receiver loop executed by the receiver worker thread.
TimerLoop(): Looper for timer worker thread. Responsible for handling time-outs and retransmissions.
bool SendFixed(int dstMachine, int dstMailbox, const char *data, int nBytes): Send fixed sized packet for data that can fit in a single packet.
int ReceiveFixed(char *outBuffer, int outCapacity, int *outSrcMachine = 0, int *outSrcMailbox = 0): Consumes one packet from the fixed length queue, if the queue is empty it will block due to how the SynchList object is implemented.

bool SendVar(int dstMachine, int dstMailbox, const char *data, int nBytes): Sends a variable sized message sequence.
void ReceiveVar(char **outData, int *outLen, int *outSrcMachine = 0, int *outSrcMailbox = 0): Receives a variable message sequence and saves it to a queue.

rnetmanager:
bool RNetManager::Valid(int h) const: helper function to lookup the handler if it is valid for use or not
int RNetManager::Socket(int localMailbox): create a new handler and tie it to a new reliable post object
int RNetManager::Close(int h): Cleans the entry from the handles table
int RNetManager::SendFixed(int h, int dstM, int dstB, int userBuf, int len): Wrapper around ReliablePost::SendFixed that uses a socket descriptor and handles data transfer from userspace to kernel space
int RNetManager::RecvFixed(int h, int userBuf, int cap, int userSrcM, int userSrcB): Wrapper around ReliablePost::ReceiveFixed that received a packet of fixed sized data and transfers the data to the user buffer
int RNetManager::SendVar(int h, int dstM, int dstB, int userBuf, int len): Wrapper around ReliablePost::SendVar that sends data from userspace as chunked data
int RNetManager::RecvVarBegin(int h, int userOutLen, int userSrcM, int userSrcB): Wrapper around ReliablePost::ReceiveVar, receives data from network to kernel buffer and returns metadata to user space (length, src machine, src box).
int RNetManager::RecvVarRead(int h, int userBuf, int cap): copies received data from kernel buffer to user buffer
int RNetManager::RecvVarEnd(int h): frees the data buffer in the kernel and cleans the handle data structure a bit

synchconsole:
void SynchConsole::SynchPutChar(const char ch): Synchronously puts a character to the console.
bool SynchConsole::feof(): Attempts to detect characters that cause integer overflows.
int SynchConsole::SynchGetChar(): Synchronously gets a character from the console.
void SynchConsole::SynchPutString(const char s[]): Synchronously puts a string to the screen.
void SynchConsole::SynchGetString(char *s, int n): Synchronously gets a string from the screen
void SynchConsole::SynchPutInt(int n): Synchronously puts an integer to the screen.
void SynchConsole::SynchGetInt(int *n): Synchronously gets an integer from the screen.

int do_SemInit(int semVal): Initializes a new kernel Semaphore and returns the identifier of that semaphore. Prepend the node to the list.
Semaphore *FindSemaphore(int id): A helper to find the the semaphore via the id
int do_SemWait(int sem_id): a handler to wait on the semaphore
int do_SemPost(int sem_id): a handler to post on the semaphore
int do_SemDestroy(int sem_id): Destroyer for the entry requested

helpers:

bool Helpers::CopyStringFromUser(int userAddr, char *kernelBuff, int maxLen): copies a string from user buffer to kernel buffer.

Beta testing branch (part6):
void StartMigratedUserProcess(int arg): Sets up the machine state for the newly migrated process to start.

migrationmanager:
void MigrationManager::RxTrampoline(int arg): a trampoline to execute RxLoop() internal function in a thread
void MigrationManager::SendWakeupToSelf(): a hacky function that sends a wakeup message to the receiver thread to unblock it in case it was waiting
void MigrationManager::RxLoop(): Looper service that listens for incoming migrated processes.

migrate:
int do_Migrate(int dstMachine): syscall backend that executes a process migration request.

migrateproto.h: MigBegin structure that sets up metadata to be sent as serialized message during migration of a process.

Automatic heap initialization of processes:

In **start.S**, the very first thing that **__start** was executing was **main** directly. We could make use of that calling convention and create a stub that executes a function that sets up the heap for the allocators before executing main. We did that by creating the **libstd.h** library:

In **start.S**:

```
.weak   user_init

       .globl __start
       .ent    __start
__start:
       //jal   main
       jal     premain // jump to stub that will setup process before main
       move    $4,$0
       jal     Exit     /* if we return from main, exit(0) */
       .end __start


       .globl  premain
       .ent    premain
premain:
       jal     user_init
       nop
       jal     main
       move    $4,$0
       j Exit
       .end    premain
```

In **libstd.c/.h**:

```
#include "mem_alloc.h"
```

```c
#include "liballoc.h"
#include "libthread.h"

//extern int main(void);

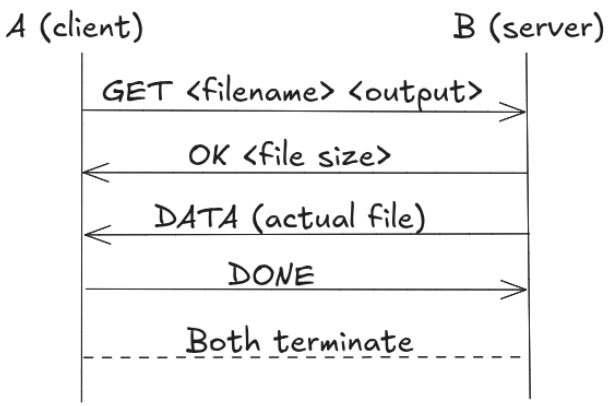
// stub to do pre-main setup, for now just setup process heap
void user_init(void){
	mutex_init(&mutex_liballoc);
	memory_init();
}
```

libstd.h userspace library:
A userspace library that contains a stub to launch heap initialization functions related to the memory allocators.

common.h: common definitions used by libraries like **NULL** and **nullptr**

Networking:

File Transfer: The simple protocol we implemented relies on simply requesting a file and then waiting for a response more in a stateless manner more like how HTTP works. The client first sends a command "GET <filename> <output filename>" to the server and waits for the server to report back the size of the file in a command "OK <file size>". The client then will be able to allocate a memory chunk to use for storing the response, and then writes it to the destined file. At the end the client sends a "DONE" command to tell the server about the termination of the connection. The diagram below illustrates this process:



Process migration:

